

SSV635X 驱动移植用户指南

1 驱动编译

1.1. 驱动可单独编译

请参考《linux ssv635x 驱动快速编译方法.txt》，建议优先采用快速编译这样可以提高移植效率。

```

#!/bin/bash
SSV_DRV_PATH=/opt/share/jinsong.yan/sstar/sstar_sdio/SSC335/ssvdriver/L.SMAC.20Q2.2102.02
KERNEL_OBJ_PATH=/opt/share/jinsong.yan/sstar/sstar_sdio/SSC335/kernel
#INSTALL_PATH=kernel/drivers/net/wireless/ssv6200
PLATFORM_TOOLCHAIN=/opt/bin/arm-buildroot-linux-uclicbnueabihf-
PLATFORM_ARCH=arm

make -C $SSV_DRV_PATH KERNEL_OBJ_PATH=$KERNEL_OBJ_PATH SSV_DRV_PATH=$SSV_DRV_PATH ARCH=$PLATFORM_ARCH CROSS_COMPILE=$PLATFORM_TOOLCHAIN clean
make -C $SSV_DRV_PATH KERNEL_OBJ_PATH=$KERNEL_OBJ_PATH SSV_DRV_PATH=$SSV_DRV_PATH ARCH=$PLATFORM_ARCH CROSS_COMPILE=$PLATFORM_TOOLCHAIN modules
make -C $SSV_DRV_PATH SSV_DRV_PATH=$SSV_DRV_PATH CROSS_COMPILE=$PLATFORM_TOOLCHAIN strip
#make -C $SSV_DRV_PATH KERNEL_OBJ_PATH=$KERNEL_OBJ_PATH SSV_DRV_PATH=$SSV_DRV_PATH INSTALL_PATH=$INSTALL_PATH install

```

如上图脚本所示，只需要设置好 4 个 shell 参数即可以直接运行脚本编译出 ko 文件；



```
build_ssv.sh
```

如图所示是快速编译 t31 平台上的示例截图

[illegible]

如编译 linux（非 andriod），建议用如下快速编译 wifi 驱动的方式，只需修改两个文件(build_ssv.sh 和 Makefile)。

快速编译 wifi 驱动 ko 简要说明。

l. 修改 build ssv.sh。

SSV_DRV_PATH 为驱动目录绝对路径

KERNEL_OBJ_PATH 为 kernel 路径

PLATFORM_TOOLCHAIN=arm-anykav200-linux-uclibcgnueabi- 替换为编译的 toolchain 前缀

PLATFORM_ARCH=arm 改为平台的 arch，一般是 arm 或 mips，例如君正平台是 mips

II. 驱动加载。

a. ssv6x5x ko 驱动加载之前，硬件连接好后请确保系统有看到 sdio 设备或 usb 设备（分别对应 ssv 的 wifi 设备）。否则要检查下 wifi 硬件或者主控平台的 sdio 或 usb 接口配置，具体可参考具体见<驱动移植用户指南.doc>相关内容。

b. 把编译好的 ssv6x5x.ko 以及驱动目录下的 ssv6x5x-wifi.cfg copy 到硬件板子上，

c. 加载驱动命令 insmod ssv6x5x.ko stacfgpath=/xxx/xxx/ssv6x5x-wifi.cfg （请不要忘记带参数，注意 635x 系列的驱动需要依赖 cfg80211.ko 所以加载驱动前确保内核 buildin cfg80211 或者已经 insmod 过 cfg80211，ubuntu 的新版本加载前需要 modprobe cfg80211,否则会有符号链接错误）

d. sdio wifi (6158/6358)和 usb wifi (6115/6355) 的 ssv6x5x-wifi.cfg 里的关于晶振配置会有不同。

1.2. Linux 内核中编译

增加 kernel 里 ssv6x5x 驱动

将 ssv6x5x 驱动存放在 drivers/net/wireless/路径下。

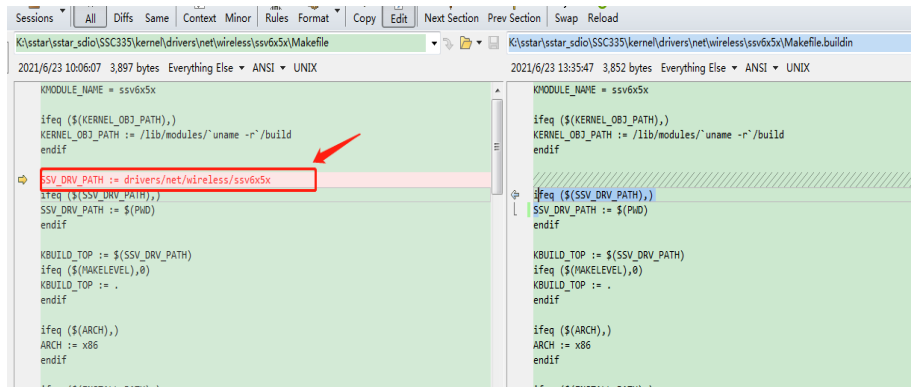
- 修改当前目录的 Makefile

```
obj-$(CONFIG_BCMDHD_AP6181) += bcmhdh_ap6181/
obj-$(CONFIG_MT7601_STA) += mt7601u/
obj-$(CONFIG_SSV6X5X) += ssv6x5x/
```

- 修改当前目录的 Kconfig

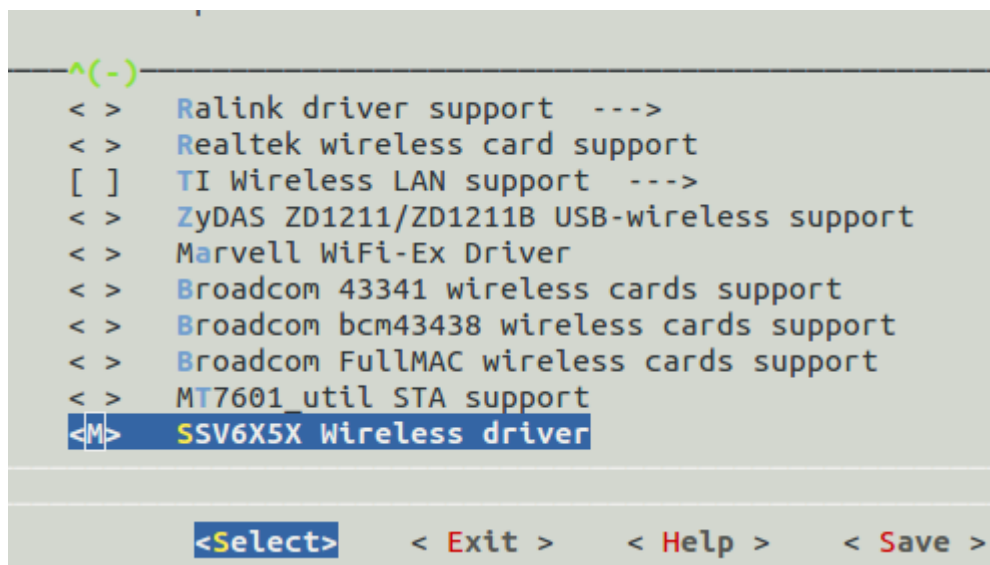
```
source "drivers/net/wireless/bcmdhd_1_141_66/Kconfig"
source "drivers/net/wireless/bcmdhd_ap6181/Kconfig"
source "drivers/net/wireless/mt7601u/Kconfig"
source "drivers/net/wireless/ssv6x5x/Kconfig"
```

- 针对 SSV6x5x 的支持，驱动里的 Makefile 增加一行



配置内核

1. ssv6x5x WiFi 驱动配置。config 需要进行配置如下。如要 ssv6x5x buildin 到内核（前提是 cfg80211 都需要 buildin 到内核），buildin 选 Y，编成 module 选 M



- 2, 因为是 Full MAC 类型驱动, 不需要内核支持 MAC80211, 内核只需支持 cfg80211 即可, buildin 选 Y, 编成 module 选 M。

```

--- Wireless
<M> cfg80211 - wireless configuration API
[*] nl80211 testmode command
[*] enable developer warnings
[*] cfg80211 certification onus
[*] enable powersave by default
[*] cfg80211 DebugFS entries
[*] use statically compiled regulatory rules database
[*] lib80211 debugging messages
<M> Generic IEEE 802.11 Networking Stack (mac80211)
[*] Minstrel
[*] Minstrel 802.11n support
[*] Minstrel 802.11ac support
[*] Default rate control algorithm (Minstrel) ---->
[*] Enable mac80211 mesh networking (pre-802.11s) support
[*] Enable LED triggers
[*] Export mac80211 internals in DebugFS
[*] Trace all mac80211 debug messages
[*] Select mac80211 debugging features ----

```

驱动 buildin 到 kernel 注意事项

1. 部分平台需要注释如下，否则驱动加载时无入口函数。

```

session-Serial-COM50-20220728192913.log ssv_firmware_version.h dev.c new 1 confi
20
21 int ssv_initWlan(void)
22 {
23     int ret=0;
24     printk(KERN_INFO "wlan.c ssv_initWlan\n");
25     ret = ssv_device_init();
26     return ret;
27 }
28
29 void ssv_exitWlan(void)
30 {
31     ssv_device_exit();
32     return;
33 }
34
35 int ssv_generic_wifi_init_module(void)
36 {
37     return ssv_initWlan();
38 }
39
40 void ssv_generic_wifi_exit_module(void)
41 {
42     ssv_exitWlan();
43 }
44
45 EXPORT_SYMBOL(ssv_generic_wifi_init_module);
46 EXPORT_SYMBOL(ssv_generic_wifi_exit_module);
47
48 #ifdef 0 //CONFIG_SSV6X5X //CONFIG_SSV6XXX=y
49     late_initcall(ssv_generic_wifi_init_module);
50 #else //CONFIG_SSV6XXX=m or =n
51     module_init(ssv_generic_wifi_init_module);
52 #endif
53     module_exit(ssv_generic_wifi_exit_module);
54

```

2. 驱动 buildin 到内核里的话，驱动运行会使用驱动 code 默认的配置参数。
并会打印这句： Use default ssv configuration

```

221 static void _ssv_import_default_cfg (char *stacfgpath)
222 {
223     struct file *fp = (struct file *) NULL;
224     char buf[MAX_CHARS_PER_LINE], cfg_cmd[32], cfg_value[32];
225     #if (LINUX_VERSION_CODE < KERNEL_VERSION(5, 10, 0))
226         mm_segment_t fs;
227     #endif
228     size_t s, read_len = 0, is_cmd_support = 0;
229     printk("\n*** %s, %s ***\n\n", __func__, stacfgpath);
230
231     // Init the buffer with 0
232     memset(&ssv_cfg, 0, sizeof(ssv_cfg));
233
234     // set default config value
235     _set_initial_cfg_default();
236
237     _set_initial_rftool_cfg_default();
238
239     if (stacfgpath == NULL)
240     {
241         //WARN_ON(1);
242         printk("Use default ssv configuration\n");
243         return;
244     }
245
246     memset(buf, 0, sizeof(buf));
247     fp = filp_open(stacfgpath, O_RDONLY, 0);

```

也可以修改驱动 code，ssvdevice.c 里的全局变量指定默认的 cfg 路径（前提是驱动加载前时文件系统可用）。

```
static char *stacfgpath = /xxx/xxx/ssv6x5x-wifi.cfg;
```

如果 stacfgpath 未指定驱动是用 code 默认的配置。见下

```

struct ssv6xxx_cfg_cmd_table cfg_cmds[] = {
    { "hw_mac", (void *)&ssv_cfg.maddr[0], 0, __string2mac, NULL },
    { "hw_cap_ampdu_rx", (void *)&ssv_cfg.hw_caps, 0, __string2flag32, "on" },
    { "hw_cap_ampdu_tx", (void *)&ssv_cfg.hw_caps, 1, __string2flag32, "on" },
    { "scan_period", (void *)&ssv_cfg.scan_period, 0, __string2u32, "50" },
    { "firmware_path", (void *)&ssv_cfg.firmware_path[0], 0, __string2str, NULL },
    { "firmware_name", (void *)&ssv_cfg.firmware_name[0], 0, __string2str, "ssv6x5x-sw.bin" },
    { "firmware_choice", (void *)&ssv_cfg.firmware_choice, 0, __string2u32, "1" },
    { "hci_trigger_en", (void *)&ssv_cfg.hci_trigger_en, 0, __string2u32, "0" },
    { "hci_trigger_glen", (void *)&ssv_cfg.hci_trigger_glen, 0, __string2u32, "10" },
    { "hci_idle_period", (void *)&ssv_cfg.hci_idle_period, 0, __string2u32, "5" },
    { "flowctl", (void *)&ssv_cfg.flowctl, 0, __string2u32, "1" },
    { "flowctl_low_threshold", (void *)&ssv_cfg.flowctl_low_threshold, 0, __string2u32, "128" },
    { "flowctl_high_threshold", (void *)&ssv_cfg.flowctl_high_threshold, 0, __string2u32, "256" },
    { "fw_reset", (void *)&ssv_cfg.fw_reset, 0, __string2u32, "0" },
    { "fw_status_idle_cnt", (void *)&ssv_cfg.fw_status_idle_cnt, 0, __string2u32, "50" },
    { "filter_duplicate_rx", (void *)&ssv_cfg.filter_duplicate_rx, 0, __string2u32, "1" },
    { "ignore_firmware_version", (void *)&ssv_cfg.ignore_firmware_version, 0, __string2u32, "0" },
    { NULL, NULL, 0, NULL, NULL },
};

```

2 驱动运行

2.1 硬件和配置检查

确认软件是否识别到 wifi ic

对于 usb 6355 wifi，执行如下确认（for 6355）

lsusb

```
Bus 001 Device 002: ID 8065:6011
```

对于 sdio wifi, 执行如下确认(for 6358)

```
cat /sys/bus/mmc/devices/mmc1\:0001/mmc1\:0001\:1/vendor
0x5356
```

注意:

1. 如果以上信息无法获取, 请确认 wifi 硬件的电源和 LDO_EN 管脚是否正常。
2. 硬件检查 ok 还是认不到 sdio vendor id, 有些平台的 mmc 或 sdio 驱动, 需要手动调用 api 重新认 sdio。例如某平台需在 ko 加载入口里添加 wifi_card_detect 或 sdio rescan 后, 就可以认到 sdio。

2.2 检测 ssv6x5x.ko 是否安装成功

- 复制相关文件到板子的文件系统中
 - 驱动目录中 ssv6x5x-wifi.cfg 文件, 例如 copy 到板子如下位置
[/etc/firmware/]

ssv6x5x-wifi.cfg 文件配置介绍

firmware_choice 这个参数非常重要, 目前驱动里面会有两个 firmware 的 h 文件供驱动通过 host hci 的通道 load 到芯片上去运行, 默认 firmware_choice = 1, 代表的是正常的工作模式, 如果需要再嵌入式系统上测试非信令的 rf 测试工作, 需要在 ssv6x5x-wifi.cfg 里面添加这个配置项, 设置值为 firmware_choice = 2 后才能进行正常的 rf 测试, 具体的 rf 测试说明件《南方硅谷 SV635x(Linux)射频测试指南_v1.1.pdf》

加载驱动 ssv6x5x.ko

- 执行如下指令: (stacfgpath=路径请根据实际路径填写)

```
insmod ssv6x5x.ko stacfgpath=/etc/firmware/ssv6x5x-wifi.cfg
```

执行 dmesg 或 cat /dev/kmsg 确认 或 打开 kernel log。调整 log 级别为最高。echo 7 >

```
/proc/sys/kernel/printk
```

可以看到如下信息, 表示驱动运行成功。


```
-E5440 kernel: [18657.663495] *** _ssv_import_default_cfg, ssv6x5x-wifi.cfg ***
-E5440 kernel: [18657.663495]
-E5440 kernel: [18657.664669] ssv6xxx_usb_init
-E5440 kernel: [18657.664698] usbcore: registered new interface driver SSV6XXX_USB
-E5440 kernel: [18657.664699] ssv6xxx_sdio_init
-E5440 kernel: [18657.724766] mmcl: new high speed SDIO card at address 0001
-E5440 kernel: [18657.724880] =====
-E5440 kernel: [18657.724881] == RUN SDIO ==
-E5440 kernel: [18657.724881] =====
-E5440 kernel: [18657.724882] vendor = 0x5356 device = 0x6030
-E5440 kernel: [18657.724883] max block count: 511
-E5440 kernel: [18657.724924] ssv6xxx_set_sdio_clk: set sdio clk 250000000Hz
-E5440 kernel: [18657.744803] sdhci-pci 0000:01:00.0: Clear PCI-E device and its parent link state L0S and L1 and CLKPM.
-E5440 kernel: [18657.744810] sdhci-pci 0000:03:00.0: Clear PCI-E device and its parent link state L0S and L1 and CLKPM.
-E5440 kernel: [18657.744811] ssv6xxx_sdio_power_on
-E5440 kernel: [18657.763150] SSV6XXX_SDIO mmcl:0001:1: dataIOPort 0x10000 regIOPort 0x10020
-E5440 kernel: [18657.783392] SSV6XXX_SDIO mmcl:0001:1: dataIOPort 0x10000 regIOPort 0x10020
-E5440 kernel: [18657.791368] wait... 0xa4221040
-E5440 kernel: [18657.791709] ssv6xxx_sdio_irq_trigger
-E5440 kernel: [18657.792099] ssv6xxx_set_sdio_clk: set sdio clk 250000000Hz
-E5440 kernel: [18657.811958] Writing firmware to SSV6XXX...
-E5440 kernel: [18657.815102] Writing firmware to SSV6XXX...
-E5440 kernel: [18657.834355] checksum = 0x522da410
-E5440 kernel: [18657.834357] ssv6xxx_set_sdio_clk: set sdio clk 330000000Hz
-E5440 kernel: [18658.351548] no mac in efuse
-E5440 kernel: [18658.351549] Apply mac address from ssv6x5x-wifi.cfg
-E5440 kernel: [18658.351552] mac address 00:62:ae:56:1e:34/00:62:ae:56:1e:35
-E5440 kernel: [18658.351605] SSV6XXX HCI TX Task started.
-E5440 kernel: [18658.351619] SSV6XXX RX Task started.
-E5440 kernel: [18658.351650] ssv6xxx_sdio_irq_disable
-E5440 kernel: [18658.351692] ssv6xxx_sdio_irq_trigger
-E5440 kernel: [18658.351735] ssv6xxx_sdio_irq_enable
-E5440 kernel: [18658.351790] [ssv_ipc_init][167]
```

进去probe 函数，能进到这里说明设备和驱动都装备好了，如果走不到这里，说明系统识别不到sdio设备或者usb 设备

这个checksum 可以真实的区别不同的驱动fw版本；

```
ade-E5440 kernel: [18895.241582]
ade-E5440 kernel: [18895.241582] sys_feat = 0x018067dd, phy_feat = 0x5de24111
ade-E5440 kernel: [18895.241582]
ade-E5440 kernel: [18895.241582]
ade-E5440 kernel: [18895.241587] ieee80211 phy3: PHY features: [NSS=1] [CHBW=40] [LDPC]
ade-E5440 kernel: [18895.241588] ieee80211 phy3: FW features: [BCN] [PS] [UAPSD] [AMPDU] [UMAC] [VHT] [MFP]
ade-E5440 kernel: [18895.241589] --->ht_on=1
ade-E5440 kernel: [18895.241592] cmd ffff8ef291412420: tkn[2] flags:001a result: -4 cmd:5120-ME_CONFIG_REQ
ade-E5440 kernel: [18895.243315] cmd ffff8ef291412060: tkn[3] flags:001a result: -4 cmd:5122-ME_CHAN_CONFIG_REQ
ade-E5440 kernel: [18895.245002] ssv wiphy phy3
ade-E5440 kernel: [18895.245004] [ssv_interface_add][313]
ade-E5440 kernel: [18895.245011] headroom = 88
ade-E5440 kernel: [18895.245286] [ssv_interface_add][313]
ade-E5440 kernel: [18895.245290] headroom = 88
ade-E5440 kernel: [18895.245495] [ssv6xxx_fmac_init][522]
ade-E5440 kernel: [18895.245499] ieee80211 phy3: New interface create p2p0
ade-E5440 kernel: [18895.245505] SSV host driver version 847, Build Data 2022-07-13
ade-E5440 kernel: [18895.245669] SSV firmware version 10229
ade-E5440 kernel: [18895.248987] =====INIT SSV CTL GENERIC NETLINK MODULE
ade-E5440 kernel: [18895.248994] =====INIT SSV WPAS CTL GENERIC NETLINK MODULE
ade-E5440 kernel: [18895.249004] SSV6XXX of iComm-semi
ade-E5440 kernel: [18895.249005] ssv_custom_modify_rf_conf_table
ade-E5440 kernel: [18895.249006] =====
ade-E5440 kernel: [18895.249007] Signature: SSV
ade-E5440 kernel: [18895.249008] Version: 0x1
ade-E5440 kernel: [18895.249008] =====
ade-E5440 kernel: [18895.249009] LT2 Config: 13 13 13 13 13 13 13 13 141 141
ade-E5440 kernel: [18895.249014] LT1 Config: 16 16 16 16 16 16 16 16 141 141
ade-E5440 kernel: [18895.249018] RT Config: 18 18 18 18 18 18 18 18 135 135
ade-E5440 kernel: [18895.249022] HT1 Config: 19 19 19 19 19 19 19 19 141 141
ade-E5440 kernel: [18895.249026] HT2 Config: 20 20 20 20 20 20 20 20 165 165
ade-E5440 kernel: [18895.249030] =====
```

用来真正的区别firmware的版本

如果 log 里没有出现 “TURISMO - USB” 或者 “RUN SDIO” 的字符，说明系统没有认到usb 或 sdio wifi，请参考上面的说明检查主控制系统软硬件设置，再次强调认不到sdio 或 usb 的情况，和驱动代码和cfg 本身无关。

- 如上述已成功则执行：ifconfig wlan0 up

检查驱动 log 是否有异常的打印。

- 如果运行正常 ifconfig -a 可看到 wlan0 和 p2p0 接口已经生成。

3 安卓系统的上层修改

请参考具体平台的移植说明。如需帮助，请与 wifi 原厂联系。

4 问题与解答

4.1. Station 模式

启动 wpa_supplicant

根据配置文件启动 wpa_supplicant 进程发起连线

```
wpa_supplicant -iwlan0 -Dnl80211 -c/mnt/wpa_supplicant.conf &
```

建议 wpa_supplicant 启动采用 nl80211，不建议使用 wext

扫描 AP

Wlan0 启动

```
ifconfig wlan0 up #wlan0 up
```

如用 iwlist 工具

```
iwlist wlan0 scan #wlan0 扫描
```

如用 wpa_cli 工具

```
wpa_cli ifname=wlan0 scan #wlan0 扫描
```

无密码连接

如用 iwconfig 工具

```
iwconfig wlan0 essid ChinaNet #连接无密码的 ESSID，为了测试专门设置一个无密码的。
```

```
iwconfig #查看是否连接上
```

如用 wpa_cli 工具

```
wpa_cli add_network #增加连线
```

```
wpa_cli set_network 0 ssid #配置 ssid
```

```
wpa_cli set_network 0 key_mgmt NONE #配置无密码加密
```

```
wpa_cli enable_network 0 #启动连线
```

```
wpa_cli status #查看当前连线状态
```

加密连接

```
killall wpa_supplicant #杀死上次运行的 wpa_supplicant 进程
```

```
wpa_passphrase ${SSID} ${PASSWD} >> /mnt/wpa_supplicant.conf #生成加密 ssid 的配置文件
```

```
wpa_supplicant -iwlan0 -Dnl80211 -c/mnt/wpa_supplicant.conf & #根据配置文件启动 wpa_supplicant 进程发起连线
```


获取 ip 地址

```
udhcpc -i wlan0&      #获取 IP 地址
busybox ifconfig      #查看 ip 地址
ping 网关 ip          #ping 网关 ip, 测试网络
```

4.2 AP 模式说明

如果需要对 softap 模式支持 wifi6 模式，对内核是有要求的，根据内核自身对 wifi6 的支持默认是需要到 4.20 之后的版本才能支持，小于 4.19 的版本，都需要针对 cfg80211 打 patch 才能支持 wifi6

组织	新建	打开	选择	
10.10.20.32	smac	turismo	sdk	6358
LFWB.23Q1.0000.00				kernel_patch
名称	修改日期	类型	大小	
3.4.110	2023/1/10 14:59	文件夹		
3.10.14	2023/5/12 10:03	文件夹		

驱动源码内目前包含了 3.4 和 3.10 针对 wifi6 的 patch 支持；至于 4.9 的内核 patch 需要对比 3.10 的修改点对比 4.20 做修改，目前驱动暂未提供；